Academic Year: 2025/26
Subject: Web Project
Group: `choosy-friends-app`

Deliverable 1 — Choosy App

1 GitHub Public Address

https://github.com/choosy-friends-app/choosy-app

The code is available in this GitHub repository created specifically for this purpose. The repository is publicly accessible without any authentication. Each participant member of the team has contributed using a different GitHub username, as requested.

2 Design Decisions

2.1 Proposed Scenario

The Choosy application solves the problem of collective decision-making in groups (leisure plans, restaurants, activities). The model reflects a complete workflow: a group of people proposes options, votes on them, and the final decision is saved as history.

2.2 Data Model

Six entities have been implemented with complex relationships:

- **Group** – Central entity with state (planning/voting/active/archived), hero image, interests (JSON field), and mission.
- **GroupMember** – M2M relationship between User and Group, enriched with role (owner/admin/member) and invitation status (invited/active/left). Allows members without a registered account.
- **PlanProposal** – Temporal voting round. Restricted to only one open proposal per group (conditional unique constraint).
- **Plan** – Individual option within a proposal. Includes price, duration, tag, location, and image.
- **Vote** – Upvote/downvote system. Supports three modes of identification: authenticated member, session user, or anonymous session.
- **Notification** – Internal notification system for invitations, new plans, and decisions.

All entities inherit from `TimestampedModel` to automatically include `created_at` and `updated_at`.

2.3 Application Architecture

- Views are organized by functionality inside `core/web/` instead of a single `views.py` file, to improve maintainability.

- Shared utilities (decorators, query helpers, template enrichment) are centralized in `shared.py`.
- The frontend uses Django Template Language with reusable components, avoiding external JS frameworks.
- CSS is modular per page/functionality to prevent collisions and ease maintenance.

2.4 Authentication

It uses the built-in Django authentication system (`django.contrib.auth`) with a custom registration form (`RegisterForm`) that includes email validation.

2.5 Docker and Deployment

- The configuration automatically switches between SQLite and PostgreSQL depending on environment variables.
- Gunicorn is used as the production WSGI server.
- Database migrations are automatically executed when the container starts.

2.6 12-Factor App

The project implements the 12-factor app guidelines: configuration via environment variables, dependencies declared in `pyproject.toml` + `uv.lock`, logs to stdout, stateless processes, and dev/prod parity via Docker.

3 Grade Distribution

All members of the group have worked and contributed equally to the project. Therefore, the grades should be perfectly equal for all members of the team.

4 Instructions: Run and Deploy

To run and deploy the application using Docker, follow these steps:

1. **Clone the repository:**

```
git clone https://github.com/choosy-friends-app/choosy-app.git
cd choosy-app
```

2. **Configure environment:**

```
cp .env.example .env
```

3. **Build and start Docker containers:**

```
docker-compose up --build
```

4. **Access the application:**
   Open your web browser at http://localhost:8000.

For more detailed deployment instructions and development setup, please refer to the `README.md` file located at the root of the project.